University of Ottawa

Faculty of Engineering
School of Electrical Engineering and Computer Science

Assignment 3

Hashing and Indexing

Course CSI2132	Databases I
Academic year	2025-26
Section	A & B
Semester	Winter
Instructor	Verena Kantere
Announced	31.03.2026
Submission	12.04.2026

Every student must submit the assignment individually

Exercise 1 (20%)

Assume that there is a relation for which you want to create an extendible hash index on an attribute that takes decimal values from the domain of integers. We use a file organization that allows 4 tuples to be stored in each page. Assume that each decimal value *d* is transformed into binary and the hashing function is: $h(k) = k \bmod 32$ (and hashing is performed with the MSBs - most significant bits). Show the index after the following insertion of values (assume an initially empty index and incremental insertions):

a. (3%) Insertion of 3, 5, 7, 10
b. (4%) Insertion of 15, 31, 44
c. (5%) Insertion of 55, 63, 64, 78, 80, 92
d. (5%) Insertion of 68, 82, 86, 90, 94
e. (3%) How many pages are needed in order to store the tuples corresponding to the above values, including the pages that store the information about the directory?

Exercise 2 (15%)

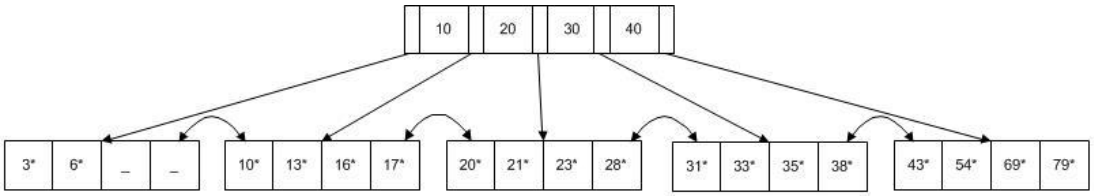
Assume that you have just built a dense B+-tree index on a heap file containing 20000 records. The key field for this B+-tree index is a 40-byte string, and it is a candidate key. Pointers are 10-byte values. The size of one disk page is 1000 bytes. The index was built in a bottom-up fashion using the bulk-loading algorithm.

a. (10%) Assume that the nodes at each level are filled as much as possible. How many levels does the B+-tree have and how many nodes does each level have?
b. (5%) How many levels would the B+-tree have if all pages are 70% full?

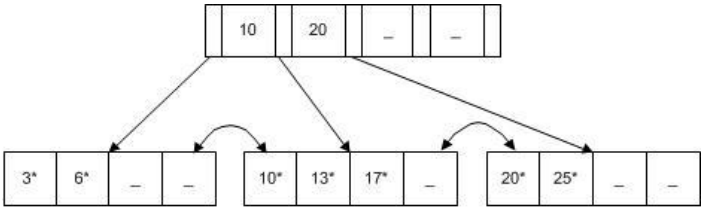
Exercise 3 (10%)

Answer the following questions concerning a B+-tree:

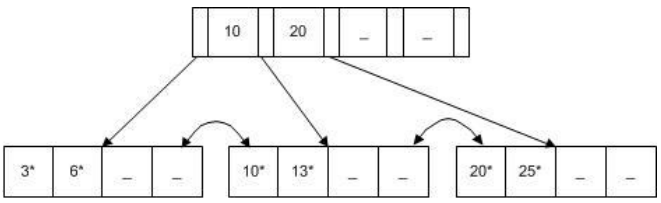
a. (4%) Consider the following B+-tree with order n = 5. Show the tree after the insertion of the value 25.



b. (3%) Consider the following B+-tree with order n = 5. Show the tree after the deletion of the value 25.

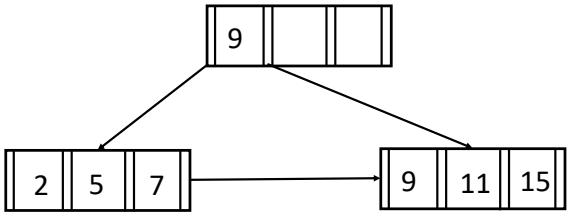


c. (3%) Consider the following B+-tree with order n = 5. Show the tree after the deletion of the value 25.



Exercise 4 (25%)

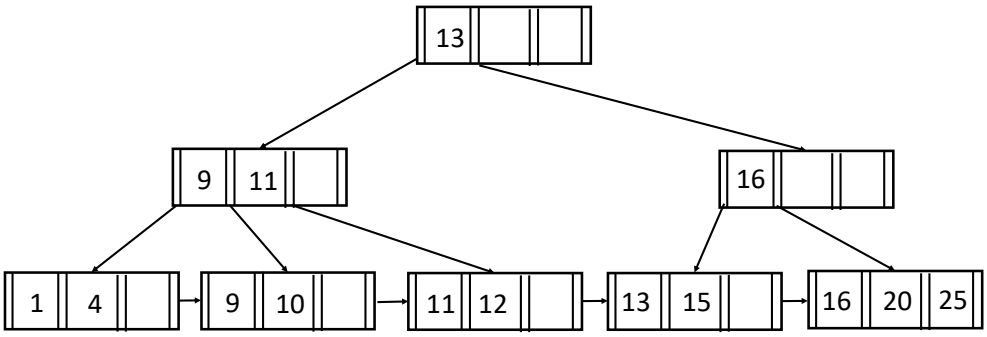
Assume there is the following B+tree index with order n = 4.



Show the tree after each one of the following actions (the actions are applied incrementally):

a. (2%) Insert 8
b. (2%) Insert 14
c. (3%) Insert 12
d. (5%) Insert 13

Assume there is the following B+tree index with order n = 4.



a. (4%) Delete 13
b. (4%) Delete 15
c. (5%) Delete 1

Exercise 5 (20%)

Suppose that we have a linear hashing index that is initially empty. At the beginning we have only two buckets (M = 2), i.e. the buckets 0 and 1, and the split pointer is pointing at bucket 0. The load factor of the index is 0.9 and splitting is performed if the load exceeds the load factor. Assume that each bucket can store 1 value. Show the index after hashing the following values:

a. (4%) 5
b. (4%) 7
c. (4%) 12
d. (4%) 11
e. (4%) 9

Exercise 6 (10%)

Assume there is a relation Student with schema Student (Name, GPA, School). The following table shows the tuples of the relation Student together with a number referring to each tuple (naturally, the 'Tuple Number' is not part of the schema – this number is there so that it can help you refer to a tuple in the creation of an index).

Tuple Number	**Name**	**GPA**	**School**
0	Phelps	4.0	ECE
1	Green	3.1	Physics
2	Thompson	2.8	ChemEng
3	Darko	3.2	Math
4	Johnson	2.0	Physics
5	Tilly	3.5	ECE
6	Thorpe	2.5	CS

a. (6%) Create a bitmap index on the attribute GPA of the relation Student, by dividing the values that appear in the column GPA in 3 ranges: assuming that X is the value, the ranges are: (i) X<2.5, (ii) 2.5<=X<3.4 and (iii) X>=3.4. Create another bitmap index on the attribute School; this index should have one entry per different value in the column School (such as 'ECE', 'Physics', etc).

b. (4%) Write in SQL the following query: "Find the names of the students in the School of 'Physics' who have a GPA equal or bigger than 3.4". Explain how this query will be executed using the bitmap indexes that you created in (a).